Virtual eXtensible LAN

Daniel Coelho Pereira
Department of Informatics Engineering
Faculty of Sciences and Technology
University of Coimbra, Portugal
uc2021237092@student.uc.pt

***Abstract*—This paper evaluates the performance overhead of the Virtual eXtensible Local Area Network (VXLAN) encapsulation protocol in a controlled single-host virtualized environment based on VMware ESXi. Two Debian virtual machines communicate over a private virtual switch, allowing the encapsulation cost to be isolated from physical-network variability. Six research questions are addressed, comparing VXLAN against direct (non-encapsulated) traffic across TCP and UDP throughput, segment size, MTU, parallel stream scalability, and CPU utilization. Results show that software-based VXLAN encapsulation reduces single-stream TCP throughput by 32.4% and increases CPU cost by 23.8% per Gbps, while adding only 0.060 ms of latency. UDP packet loss rises from 0.12% to 8.43% under VXLAN, exposing the asymmetric impact of encapsulation on connectionless traffic. Enabling jumbo frames reduces the overhead to 18.0%, and with multiple parallel streams VXLAN outperforms direct traffic by up to 105%, attributable to Receive Side Scaling in the VMware vSwitch. Disabling sender-side NIC offloads causes complete VXLAN TCP connection failure, demonstrating a critical dependency on GSO and TSO for software VTEP operation. All test scripts and raw results are publicly available at https://github.com/danielcpereira/vxlan-performance-analysis.**

***Index Terms*—VXLAN, network virtualization, overlay networks, Layer-2 tunneling, encapsulation overhead, performance evaluation.**

I. Introduction

Modern cloud and edge data centers consolidate large numbers of virtual machines and containers, belonging to many tenants, onto shared physical infrastructure. Network virtualization decouples the logical connectivity of these workloads from the underlying physical topology, enabling tenant isolation, transparent workload mobility, and elastic scaling. For two decades this separation was provided primarily at Layer 2 by IEEE 802.1Q VLANs, which were not designed for the scale and dynamism of contemporary virtualized environments [1].

The 12-bit VLAN identifier limits a switched fabric to 4094 usable segments, which is insufficient for large multi-tenant deployments. In addition, VLANs are commonly deployed using a one-subnet-per-VLAN design and operated in conjunction with the Spanning Tree Protocol in traditional Layer 2 networks, limiting both host mobility and efficient link utilization. Overlay networks address these constraints by tunneling Layer 2 frames across a routed Layer 3 underlay. Among the proposed encapsulation mechanisms, the Virtual eXtensible Local Area Network (VXLAN) has become the dominant standard. VXLAN encapsulates Ethernet frames in UDP (MAC-in-UDP) and introduces a 24-bit segment identifier, increasing the number of addressable segments to approximately sixteen million [2].

This flexibility introduces additional overhead. Each frame transmitted through a VXLAN tunnel carries 50 extra bytes of encapsulation headers and requires encapsulation and decapsulation processing at the tunnel endpoints, operations that are often handled in software by the host networking stack. Consequently, VXLAN may reduce throughput, increase CPU utilization, and add latency. The impact of this overhead varies according to factors such as the transport protocol, MTU, packet size, and traffic concurrency. Accurate characterization of these effects is important for capacity planning and requires controlled experimental conditions to avoid interference from physical network variability.

To quantify this overhead under controlled conditions, this paper provisions, configures, and experimentally evaluates a VXLAN overlay in a single-host virtualized environment based on VMware ESXi, in which two Debian virtual machines communicate over a private virtual switch with no physical uplink. Confining the experiment to a single host eliminates physical-network variability and isolates the cost of the encapsulation itself. The evaluation is organized around a set of research questions and compares VXLAN against direct (non-encapsulated) traffic using `iperf3`, for both TCP and UDP, across a range of MTUs, packet sizes, and parallel streams. Beyond throughput, latency, jitter, and packet loss, the study also measures the CPU cost of encapsulation and its sensitivity to NIC offloading, since VXLAN processing is performed in software. Each configuration is measured over multiple repetitions and reported as a mean with its associated dispersion.

The remainder of this paper is organized as follows. Section II reviews the state of the art in Layer 2 tunneling protocols and overlay encapsulation mechanisms, and positions VXLAN among them. Section III describes the testing environment, and Section IV details the VXLAN provisioning and configuration procedure. Section V presents the evaluation methodology, and Section VI reports and discusses the experimental results. Finally, Section VII concludes the paper and discusses the limitations of the study.

II. Background and State of the Art

A. Ethernet and the Limits of Classic VLANs

Ethernet is the dominant Layer 2 technology in data-center networks. To partition a single physical Ethernet fabric into multiple isolated broadcast domains, IEEE 802.1Q inserts a 4-byte tag into the Ethernet header, of which a 12-bit VLAN Identifier (VID) designates the virtual LAN a frame belongs to [1]. This mechanism served enterprise networks well for two decades, but several of its properties become limiting in large, virtualized, multi-tenant environments.

First, the 12-bit VID allows at most 4094 usable segments (two values are reserved), which is insufficient when each of many tenants requires several isolated networks. Second, a VLAN is inherently bound to the physical Layer 2 topology: a given broadcast domain typically maps to a single IP subnet and cannot easily span separate Layer 3 boundaries, which constrains the placement and live migration of virtual machines across racks or hosts. Third, loop-free operation in a switched Layer 2 network relies on the Spanning Tree Protocol (STP), which disables redundant links to prevent loops; this wastes available bandwidth and yields slow reconvergence after topology changes, in contrast to the equal-cost multipath (ECMP) load balancing routinely available at Layer 3.

These limitations motivate *overlay networks*, which decouple the tenant's logical Layer 2 view from the physical network by encapsulating Ethernet frames and transporting them across a routed Layer 3 underlay. The underlay can then use ECMP and standard IP routing for scalability and resilience, while tenants retain the abstraction of a flat Layer 2 segment.

B. Layer-2 Tunneling Protocols

The idea of carrying Layer 2 frames across an intermediate network predates data-center overlays. Three protocols are historically significant: PPTP, L2F, and L2TP. All three were conceived primarily to extend Point-to-Point Protocol (PPP) sessions across an IP network for remote access and virtual private networks (VPNs), rather than to provide large-scale segmentation inside a data center.

The Point-to-Point Tunneling Protocol (PPTP) encapsulates PPP frames inside a modified Generic Routing Encapsulation (GRE) payload and uses a separate TCP control connection on port 1723 [3]. It was widely deployed in early VPN products but is now considered insecure, as the authentication and encryption schemes commonly paired with it (MS-CHAPv2 and MPPE) have known weaknesses. Cisco's Layer 2 Forwarding (L2F) protocol pursued the same goal of tunneling PPP sessions but encapsulated them directly over UDP, and delegated authentication and confidentiality to other mechanisms rather than defining its own [4].

The Layer 2 Tunneling Protocol (L2TP) combined the strengths of PPTP and L2F into a single IETF standard; it carries PPP frames over UDP and is typically paired with IPsec (L2TP/IPsec) to provide confidentiality and integrity [5]. Its successor, L2TPv3, generalizes the data plane to transport arbitrary Layer 2 frames—not only PPP—over IP, enabling Ethernet pseudowires between provider edges [6].

TABLE I
Comparison of Layer-2 tunneling protocols.

Protocol	Encapsulation	Security	Typical use case
PPTP	PPP over GRE; TCP/1723 control	Weak (MS-CHAPv2, MPPE)	Legacy remote-access VPN
L2F	PPP over UDP/1701	None native (delegated)	Early ISP-tunneled VPN
L2TP	PPP over UDP/1701	With IPsec (L2TP/IPsec)	Site-to-site / remote VPN
L2TPv3	Any L2 frame over IP	With IPsec	Ethernet pseudowires (provider edge)
VXLAN	MAC-in-UDP/4789, 24-bit VNI	None native (underlay-based)	Multipoint data-center overlay

The essential distinction from data-center overlays is architectural rather than incidental. These protocols establish *point-to-point* tunnels between two endpoints for remote-access or site-to-site connectivity, and they do not provide a scalable identifier space for tens of thousands of isolated segments or efficient multipoint, any-to-any communication among many hosts. VXLAN and the other encapsulations discussed next were designed specifically for that *multipoint*, highly scalable, intra-data-center use case. Table I summarizes the comparison.

C. VXLAN

VXLAN was standardized to provide scalable Layer 2 connectivity over a routed Layer 3 network [2]. It follows a *MAC-in-UDP* encapsulation model: a complete inner Ethernet frame, as produced by a virtual machine, is wrapped in a VXLAN header and transported as the payload of a UDP datagram between two tunnel endpoints. The encapsulation adds an 8-byte VXLAN header, an 8-byte UDP header, a 20-byte outer IPv4 header, and a 14-byte outer Ethernet header, for a total of 50 bytes of overhead per frame, as shown in Fig. 1. Because the outer packet is an ordinary IP/UDP packet, it can be forwarded by any Layer 3 device in the underlay without awareness of the overlay, and the underlay can exploit equal-cost multipath routing for load balancing.

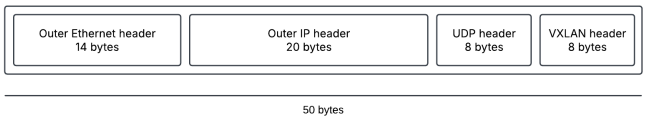

Fig. 1. VXLAN encapsulation headers added to each frame, totalling 50 bytes of overhead.

The entity that performs encapsulation and decapsulation is the VXLAN Tunnel End Point (VTEP). A VTEP has one interface in the overlay (where it presents a virtual Layer 2 segment to the attached workloads) and one interface in the underlay (with a routable IP address used as the source or destination of the outer packet). When a frame leaves a virtual machine, the local VTEP adds the outer headers and sends

the resulting packet toward the remote VTEP, which removes them and delivers the original frame unchanged. A VTEP may be implemented in a physical switch or, as in this work, in software within the host operating system.

Each overlay segment is identified by a 24-bit VXLAN Network Identifier (VNI) carried in the VXLAN header. The 24-bit field allows approximately sixteen million distinct overlay segments ($2^{24} = 16\,777\,216$) in contrast to the 4094 usable VLANs of IEEE 802.1Q; this is the property that makes VXLAN suitable for large multi-tenant environments. The destination UDP port is fixed (4789), while the source UDP port can be derived from a hash of the inner frame's headers, providing per-flow entropy so that underlay devices may distribute different flows across multiple paths.

A remaining question is how a VTEP learns which remote VTEP holds a given destination MAC address. The original specification relies on multicast: each VNI is associated with an IP multicast group, and unknown-unicast, broadcast, and multicast traffic is flooded to that group, with VTEPs learning MAC-to-VTEP associations from the encapsulated traffic they receive. Because multicast is not always available or desirable, two alternatives are common: a *static unicast* configuration, in which the forwarding database (FDB) of each VTEP is populated manually with the address of the remote endpoint, and control-plane approaches such as EVPN over BGP, which distribute MAC-to-VTEP reachability without flooding [9]. In the single-host, two-endpoint scenario used in this work, a static unicast point-to-point configuration is the simplest and most reproducible choice, as it avoids any dependency on multicast in the underlay; this configuration is detailed in Section IV.

D. Comparison of Modern Overlay Encapsulations

VXLAN is not the only encapsulation proposed for network virtualization; several alternatives emerged around the same period, each making different design trade-offs. Network Virtualization using Generic Routing Encapsulation (NVGRE) carries the inner Ethernet frame over GRE rather than UDP and uses a 24-bit Virtual Subnet Identifier, providing the same segment scale as VXLAN [7]. Its main practical drawback is the absence of a UDP source port: because GRE exposes no equivalent per-flow entropy field, underlay devices have more difficulty distributing NVGRE flows across multiple paths, which can limit load balancing.

Generic Network Virtualization Encapsulation (GENEVE) was designed to address the rigidity of fixed-format headers. Like VXLAN, it uses MAC-in-UDP encapsulation and a 24-bit identifier, but it adds a variable-length, type-length-value (TLV) options field, allowing control planes to attach metadata to packets without defining a new protocol [8]. This extensibility comes at the cost of a slightly larger and variable header, which marginally increases overhead. The Stateless Transport Tunneling (STT) proposal took a different approach, emulating a TCP-like header so that existing NIC TCP-segmentation-offload hardware could accelerate the encapsulated traffic; it saw limited adoption and was largely superseded by hardware that offloads VXLAN and GENEVE directly.

TABLE II
COMPARISON OF MODERN OVERLAY ENCAPSULATIONS.

Overlay	Transport	Segment ID	Distinguishing feature
VXLAN	MAC-in-UDP (4789)	24-bit VNI	De facto standard; UDP entropy for ECMP
NVGRE	MAC-in-GRE	24-bit VSID	No UDP port; weaker multipath support
GENEVE	MAC-in-UDP (6081)	24-bit VNI	Extensible TLV options; variable header
STT	TCP-like header	64-bit ctx	Designed for NIC TSO offload; little adoption

The differences among these encapsulations are summarized in Table II. Empirical comparisons report that the performance gap between them is small in practice. For instance, evaluations of VXLAN and GENEVE in software-defined networks find comparable throughput, latency, and loss, with GENEVE consuming marginally more bandwidth owing to its larger, flexible header [10]. Comparative studies of VXLAN and its control-plane variants similarly report that the encapsulation choice has a smaller effect on performance than the surrounding control plane and hardware offload support [11]. VXLAN remains the most widely deployed of these encapsulations, owing to broad hardware and hypervisor support, which is why it is the focus of this work.

III. TESTING ENVIRONMENT

The evaluation is conducted in a single-host virtualized environment hosted on a VMware ESXi server. Both virtual machines run Debian GNU/Linux 13 (kernel 6.12.73) and reside on the same physical host, connected exclusively through a private virtual switch (`vSwitch_DP`) with no physical uplink. This topology matches the single-host scenario recommended in the assignment specification and is illustrated in Fig. 2.

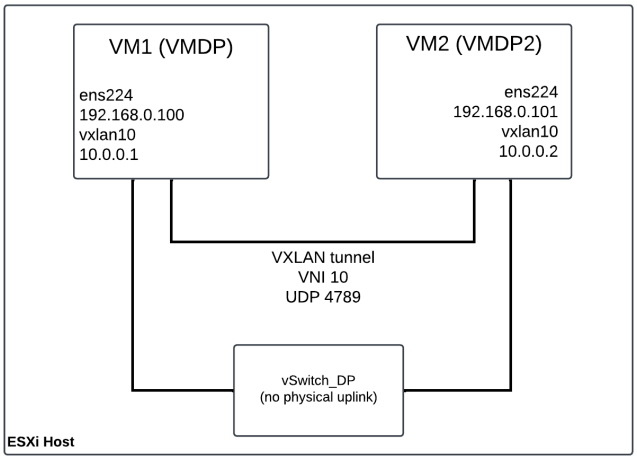

Fig. 2. Single-host VXLAN test topology

Confining both endpoints to the same host serves a deliberate methodological purpose: it eliminates physical-network variability (cable quality, switch queuing, and competing traffic) and isolates the performance impact of the VXLAN encapsulation itself. Any throughput reduction or CPU increase observed in the VXLAN tests can therefore be attributed to the overhead of encapsulation and decapsulation, rather than to the characteristics of the underlying physical network.

Table III summarizes the configuration of both virtual machines. The underlay network uses the `ens224` interface on the `192.168.0.0/24` subnet, over which the VXLAN tunnel is established. The overlay network is provided by the `vxlan10` interface, configured with a /24 subnet in the `10.0.0.0/24` range. The VXLAN endpoint (VTEP) is implemented in software by the Linux kernel on each VM, using a static unicast point-to-point tunnel configuration, as described in Section IV. Performance measurements are conducted between the two overlay IP addresses, using `iperf3` (version 3.18) for traffic generation, `mpstat` for CPU utilization monitoring, `tcpdump` for packet capture and encapsulation validation, and `ethtool` for NIC offload inspection.

TABLE III
VIRTUAL MACHINE CONFIGURATION.

Parameter	VM1 (VMDP)	VM2 (VMDP2)
Role	iperf3 server / VTEP A	iperf3 client / VTEP B
OS	Debian GNU/Linux 13	Debian GNU/Linux 13
Kernel	6.12.73-amd64	6.12.73-amd64
Underlay IP	192.168.0.100	192.168.0.101
Overlay IP	10.0.0.1	10.0.0.2
Underlay iface	ens224 (MTU 1500)	ens224 (MTU 1500)
Overlay iface	vxlan10 (MTU 1450)	vxlan10 (MTU 1450)
Management IP	10.254.0.120	10.254.0.193
vCPUs	4	4
vSwitch	vSwitch_DP (no physical uplink)	

IV. VXLAN PROVISIONING AND CONFIGURATION

This section describes the steps required to provision and configure the VXLAN overlay between the two virtual machines, starting from the underlay verification and ending with the validation of end-to-end overlay connectivity. All commands were executed directly on each VM as root. The configuration is intentionally kept stateless and reproducible: no persistent network configuration files were modified, and the overlay can be re-created from scratch by re-running the commands below.

A. Underlay Verification

Before provisioning the overlay, connectivity between the two VMs over the underlay network was verified. From VM1, a reachability test was performed toward VM2's underlay address:

```
ping -c 3 192.168.0.101
```

The test confirmed bidirectional reachability with no packet loss and a round-trip time below 1 ms, which is expected for two virtual machines sharing the same physical host and connected through a private virtual switch. This step ensures that the underlay is functional before any overlay configuration is applied.

B. VXLAN Tunnel Endpoint Configuration

The VXLAN tunnel endpoint (VTEP) was created on each VM using the native Linux kernel VXLAN driver, following the approach described in [12]. The following command was executed on VM1:

```
ip link add vxlan10 type vxlan id 10 \
    remote 192.168.0.101 \
    local  192.168.0.100 \
    dev ens224 dstport 4789
```

On VM2, the symmetric command was used, with the `remote` and `local` addresses swapped. Each parameter serves a specific purpose:

- `id 10` sets the VXLAN Network Identifier (VNI) to 10, identifying the overlay segment.
- `remote` specifies the underlay IP address of the remote VTEP, establishing a static unicast point-to-point tunnel without requiring multicast or a separate forwarding database entry.
- `local` binds the VTEP to the local underlay IP address, ensuring that encapsulated packets are sourced from the correct interface.
- `dev ens224` associates the VTEP with the underlay interface on the `192.168.0.0/24` subnet.
- `dstport 4789` sets the IANA-assigned UDP destination port for VXLAN, ensuring standard-compliant encapsulation per RFC 7348 [2].

This approach directly encodes the remote endpoint address in the tunnel definition, which is the simplest and most appropriate configuration for a two-endpoint scenario. It avoids any multicast dependency in the underlay and produces fully deterministic, reproducible forwarding behavior.

Since each virtual machine hosts a single network endpoint, no bridge interface was required; the overlay IP address was assigned directly to the `vxlan10` interface, simplifying the configuration relative to multi-endpoint deployments.

C. Overlay IP Assignment and Interface Activation

With the VTEP configured, an IP address from the overlay subnet was assigned to the `vxlan10` interface, and the interface was brought up:

```
# VM1
ip addr add 10.0.0.1/24 dev vxlan10
ip link set vxlan10 up

# VM2
ip addr add 10.0.0.2/24 dev vxlan10
ip link set vxlan10 up
```

The Linux kernel automatically sets the MTU of the `vxlan10` interface to 1450 bytes, derived from the underlay MTU (1500 bytes) minus the 50 bytes of VXLAN encapsulation overhead. No explicit MTU configuration was therefore required for the standard-MTU tests. For the jumbo-frame tests (RQ4), the underlay and overlay MTUs were adjusted accordingly, as described in Section V.

D. Overlay Connectivity Validation

After configuration, end-to-end connectivity across the VXLAN overlay was validated from both directions. From VM1:

```
ping -c 3 10.0.0.2
```

And from VM2:

```
ping -c 3 10.0.0.1
```

Both tests confirmed full bidirectional connectivity with zero packet loss. The observed round-trip time over the overlay (0.46 ms) was slightly higher than over the underlay (0.37 ms), reflecting the processing cost of encapsulation and decapsulation at the two software VTEPs, the same overhead that the subsequent performance evaluation quantifies in detail.

V. Evaluation Methodology

This section describes the experimental design used to evaluate the performance overhead of VXLAN encapsulation. The evaluation is structured around six research questions (RQs), each targeting a specific dimension of the overhead. All tests were conducted in the single-host environment described in Section III, with the VXLAN overlay configured as detailed in Section IV.

A. Research Questions

The following research questions guide the evaluation:

- **RQ1 — Baseline TCP overhead:** What is the throughput reduction introduced by VXLAN encapsulation relative to direct (non-encapsulated) TCP traffic?
- **RQ2 — TCP vs. UDP overhead:** Is the encapsulation overhead asymmetric between TCP and UDP? How does VXLAN affect packet loss and jitter under UDP?
- **RQ3 — Impact of segment size:** How does the relative overhead vary with the TCP maximum segment size (MSS)?
- **RQ4 — Impact of MTU and Jumbo Frames:** Does increasing the MTU of the underlay and overlay interfaces reduce the encapsulation overhead?
- **RQ5 — Parallel stream scalability:** Does VXLAN throughput scale linearly with the number of parallel TCP streams?
- **RQ6 — CPU overhead and NIC offloads:** What is the CPU cost of VXLAN encapsulation, and how does disabling NIC offloads affect it?

B. Test Matrix

For each research question, traffic is generated between the two overlay IP addresses (`10.0.0.1` and `10.0.0.2`) for VXLAN tests, and between the two underlay addresses (`192.168.0.100` and `192.168.0.101`) for direct baseline tests. Table IV summarizes the test parameters.

TABLE IV
Test matrix.

RQ	Protocol	Variable	Values	MTU
RQ1	TCP	Encapsulation	Direct / VXLAN	1500/1450
RQ2	TCP + UDP	Protocol	TCP / UDP	1500/1450
RQ3	TCP	MSS	256/512/1024/1400 B	1500/1450
RQ4	TCP	MTU	1500/9000 B	varied
RQ5	TCP	Streams	1/2/4/8	1500/1450
RQ6	TCP	NIC offloads	on / off	1500/1450

C. Tools and Metrics

Traffic generation and measurement use `iperf3` (version 3.18), which produces structured JSON output for automated analysis. For TCP tests, the primary metric is the received throughput in Gbps and the number of retransmissions. For UDP tests, throughput, packet loss percentage, and jitter are reported. CPU utilization is captured concurrently using `mpstat` (from the `sysstat` package), sampling at 1-second intervals throughout each test. The key CPU metric is the softirq percentage (`%soft`), which reflects the kernel processing cost of network encapsulation. Additionally, CPU efficiency is reported as `%CPU per Gbps`, normalizing the CPU cost by the achieved throughput. Packet captures using `tcpdump` were used to validate the VXLAN encapsulation and confirm the 50-byte overhead on the wire. NIC offload inspection and manipulation use `ethtool`.

D. Experimental Rigor

Each configuration is repeated five times. Results are reported as the mean with the associated standard deviation. A 5-second idle period is introduced between consecutive repetitions to allow the networking stack to return to a steady state. The `iperf3` client and `mpstat` are launched concurrently so that CPU measurements are synchronized with the traffic generation period. All tests were conducted with no other workloads running on either virtual machine. The test scripts and raw results are publicly available at: https://github.com/danielcpereira/vxlan-performance-analysis .

Baseline measurements were conducted at the start of each experimental session. The observed variation in single-stream direct throughput across sessions (11.5–13.9 Gbps) reflects scheduler and vSwitch state differences between sessions rather than measurement error.

VI. Results and Discussion

This section presents and discusses the experimental results for each research question. All throughput values are reported

as the mean over five repetitions with the associated standard deviation. CPU utilization is reported as the percentage of time spent in softirq processing (`%soft`) and as CPU efficiency (%CPU per Gbps), computed as $(100 - \%\text{idle})/\text{throughput}$. Direct (non-encapsulated) traffic serves as the baseline in all comparisons; the encapsulation overhead is defined as $(T_{\text{direct}} - T_{\text{VXLAN}})/T_{\text{direct}} \times 100\%$.

A. Latency Overhead

Table V reports the round-trip time measured by `ping` over 100 packets for both the underlay (direct) and overlay (VXLAN) paths, prior to any iperf3 traffic.

TABLE V
ROUND-TRIP LATENCY: DIRECT (UNDERLAY) VS VXLAN (OVERLAY).

Path	Min	Avg	Max	Mdev
Direct (underlay)	0.247 ms	0.401 ms	1.069 ms	0.094 ms
VXLAN (overlay)	0.320 ms	0.461 ms	0.850 ms	0.068 ms
Overhead	+0.073 ms	+0.060 ms	—	—

The VXLAN overlay adds 0.060 ms to the average round-trip time, a relative increase of 15.0%. In absolute terms this is negligible for most applications. Notably, the VXLAN path exhibits lower maximum latency (0.850 ms vs 1.069 ms) and lower variability (mdev 0.068 vs 0.094 ms) than the direct path. This suggests that the encapsulation processing introduces a small but consistent delay that paradoxically regularizes packet timing, smoothing out the occasional spikes observed in the direct path. Zero packet loss was recorded on both paths.

B. RQ1: TCP Encapsulation Overhead (Baseline)

Table VI summarizes the TCP throughput and CPU utilization for direct and VXLAN traffic under standard MTU conditions (underlay 1500 B, overlay 1450 B), with a single parallel stream.

TABLE VI
RQ1 — TCP THROUGHPUT AND CPU UTILIZATION (MTU 1500/1450).

Mode	Throughput	%idle	%soft	CPU/Gbps
Direct	13.879 ± 0.213 Gbps	82.1%	0.604%	1.290%
VXLAN	9.379 ± 0.697 Gbps	85.0%	0.705%	1.597%
Overhead			32.4%	

VXLAN encapsulation reduces TCP throughput by 32.4% relative to direct traffic (13.879 vs 9.379 Gbps). This overhead reflects the cost of software-based encapsulation and decapsulation at the two VTEPs: for every frame transmitted, the Linux kernel must construct 50 bytes of outer headers on the sending side and strip them on the receiving side, without hardware assistance. The CPU efficiency metric confirms this interpretation, VXLAN requires 1.597 %CPU per Gbps against 1.290 %CPU per Gbps for direct traffic, a difference of approximately 23.8%, meaning that the encapsulation path consumes proportionally more CPU resources per unit of useful data delivered.

The standard deviation is notably higher for VXLAN (± 0.697 Gbps) than for direct traffic (± 0.213 Gbps), indicating greater variability in the encapsulation path. This is consistent with the software nature of the VTEP implementation: encapsulation and decapsulation compete with other kernel tasks for CPU time, introducing scheduling jitter that is absent in the direct path.

The observed overhead is substantially higher than the values reported in hardware-accelerated deployments, where NIC offloads for VXLAN reduce the encapsulation cost to near-zero [14]. This difference is expected given that the present evaluation uses a software VTEP on a private virtual switch with no physical NIC in the VXLAN data path, deliberately chosen to isolate the pure encapsulation cost from hardware effects.

C. RQ2: TCP vs. UDP Overhead

Table VII compares the encapsulation overhead for TCP and UDP. For UDP, the target bandwidth was set to 20 Gbps to saturate the available capacity, and results include packet loss and jitter in addition to throughput.

TABLE VII
RQ2 — TCP VS. UDP OVERHEAD.

Proto	Mode	Throughput	Loss	Jitter
TCP	Direct	13.879 Gbps	—	—
TCP	VXLAN	9.379 Gbps	—	—
UDP	Direct	1.752 ± 0.031 Gbps	0.12%	0.019 ms
UDP	VXLAN	1.256 ± 0.018 Gbps	8.43%	0.014 ms

The UDP throughput is substantially lower than TCP in both modes (1.752 vs 13.879 Gbps for direct). A natural reading would attribute this to UDP's lack of flow control overwhelming the receiver, but the observed loss of only 0.12% in the direct case contradicts that explanation: a saturated receiver would exhibit much higher loss. The more plausible interpretation is that the iperf3 sender itself becomes the dominant bottleneck for UDP, as iperf3 is single-threaded and issues one syscall per datagram, limiting achievable throughput regardless of system capacity. The asymmetry with VXLAN supports this reading: under VXLAN the throughput drops only modestly (to 1.256 Gbps) but loss rises sharply to 8.43%, indicating that VXLAN encapsulation is the first factor in this test environment to surpass the iperf3 generation limit and reach the actual receiver capacity. The encapsulation overhead therefore manifests asymmetrically: invisible under TCP's congestion control, but exposed as loss once the receiver becomes the binding constraint.

Jitter is marginally lower under VXLAN (0.014 vs 0.019 ms), suggesting that the encapsulation path introduces a small but consistent processing delay that paradoxically regularizes inter-packet spacing.

D. RQ3: Impact of Segment Size

Table VIII and Fig. 3 show TCP throughput for direct and VXLAN traffic across four requested MSS values. An important methodological note applies: during this experiment, Generic Segmentation Offload (GSO) was active on both virtual machines. GSO aggregates outgoing TCP segments into 64 KB super-frames before passing them to the encapsulation layer, regardless of the MSS value requested via the `-M` flag. Packet captures confirmed that on-wire segment sizes were approximately 65 KB in all cases. Consequently, the results reflect the behavior of the Linux networking stack with GSO enabled, the normal production configuration, rather than the effect of the raw MSS parameter in isolation.

TABLE VIII
RQ3 — TCP THROUGHPUT VS. REQUESTED MSS (GSO ACTIVE).

MSS	Direct	VXLAN	Overhead
256 B	11.512 ± 0.417 Gbps	9.304 ± 0.347 Gbps	19.2%
512 B	11.651 ± 0.309 Gbps	8.886 ± 0.831 Gbps	23.7%
1024 B	11.817 ± 0.272 Gbps	9.420 ± 0.446 Gbps	20.3%
1400 B	11.322 ± 0.441 Gbps	8.090 ± 0.704 Gbps	28.5%

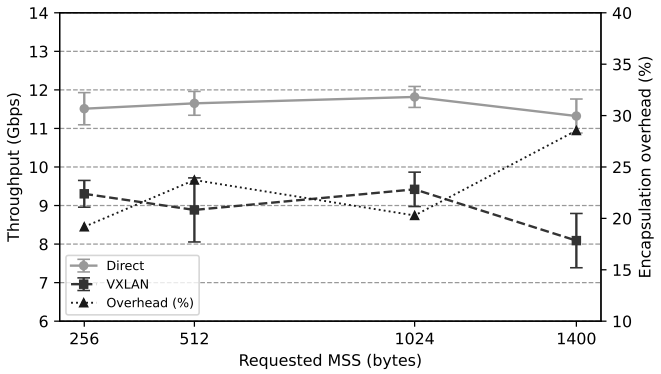

Fig. 3. RQ3 — TCP throughput and encapsulation overhead vs. requested MSS (GSO active). Error bars show one standard deviation over five repetitions.

Despite GSO masking the requested MSS, a consistent trend is observable: the encapsulation overhead increases with the requested MSS value, reaching 28.5% at MSS = 1400 B compared to 19.2% at MSS = 256 B. This behavior is attributable to the interaction between GSO and the VXLAN MTU constraint. At larger MSS values, GSO super-frames approach the VXLAN MTU limit of 1450 bytes more frequently, reducing GSO efficiency and increasing the per-byte cost of encapsulation. At smaller MSS values, GSO can aggregate more segments per super-frame, amortizing the encapsulation cost over more data.

E. RQ4: Impact of MTU and Jumbo Frames

Enabling jumbo frames (underlay MTU 9000 B, overlay MTU 8950 B) reduces the VXLAN encapsulation overhead from 32.4% to 18.0%, as shown in Table IX.

TABLE IX
RQ4 — EFFECT OF MTU ON THROUGHPUT AND OVERHEAD.

MTU	Mode	Throughput	Overhead
1500/1450	Direct	13.879 ± 0.213 Gbps	32.4%
	VXLAN	9.379 ± 0.697 Gbps	
9000/8950	Direct	12.335 ± 0.584 Gbps	18.0%
	VXLAN	10.116 ± 0.317 Gbps	

The reduction in overhead is explained by the fixed nature of the 50 byte encapsulation cost: with a larger MTU, each packet carries more payload, reducing the relative weight of the overhead from $50/1450 \approx 3.4\%$ to $50/8950 \approx 0.6\%$ per packet. Furthermore, fewer packets are required to transmit the same volume of data, reducing the total number of encapsulation operations and their cumulative CPU cost.

The direct throughput with jumbo frames (12.335 Gbps) is slightly lower than with standard MTU (13.879 Gbps). This counter-intuitive result is attributable to the VMware vSwitch internal processing path: larger frames require more memory bandwidth per packet in the virtual switch, which can reduce throughput in intra-host scenarios where the data path is memory-bound rather than network-bound.

F. RQ5: Parallel Stream Scalability

Table X and Fig. 4 show the aggregate TCP throughput as the number of parallel iperf3 streams increases from 1 to 8.

TABLE X
RQ5 — THROUGHPUT VS. NUMBER OF PARALLEL STREAMS.

Streams	Direct	VXLAN	Result
1	11.478 ± 0.369 Gbps	7.982 ± 0.774 Gbps	VXLAN 30.5% slower
2	7.191 ± 0.270 Gbps	14.762 ± 0.420 Gbps	VXLAN 105% faster
4	8.187 ± 3.568 Gbps	14.925 ± 0.558 Gbps	VXLAN 82% faster
8	11.677 ± 6.883 Gbps	14.910 ± 0.142 Gbps	VXLAN 28% faster

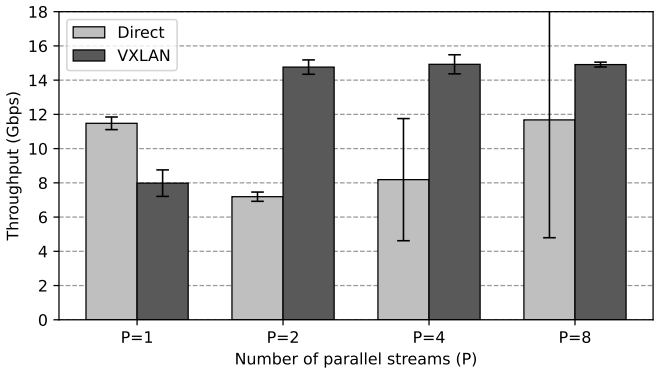

Fig. 4. Aggregate TCP throughput vs. number of parallel streams. Error bars show one standard deviation over five repetitions.

With a single stream, VXLAN is 30.5% slower than direct traffic, consistent with RQ1. With two or more parallel

streams, however, VXLAN consistently outperforms direct traffic, reaching up to 14.925 Gbps at P = 4 compared to 8.187 Gbps for direct.

VXLAN encapsulates each stream into a separate outer UDP flow, using a per-flow hash to derive distinct UDP source ports. This per-flow entropy allows the VMware vSwitch to leverage Receive Side Scaling (RSS) to distribute the processing of incoming VXLAN packets across multiple CPU cores [13].

Direct TCP streams, by contrast, compete independently for vSwitch resources, causing contention that limits scalability. Telefónica Engineering reported analogous behavior in physical deployments, observing that VXLAN traffic can outperform native traffic when overlay-specific NIC offloads are active [14].

The high standard deviation of direct traffic at P = 4 (± 3.568 Gbps) and P = 8 (± 6.883 Gbps) further confirms the instability of the direct path under concurrent streams in this virtual environment, while VXLAN remains stable (± 0.558 and ± 0.142 Gbps respectively).

G. RQ6: CPU Overhead and NIC Offloads

Table XI and Fig. 5 summarize CPU utilization and throughput for direct and VXLAN traffic, with sender-side NIC offloads enabled and disabled.

TABLE XI
RQ6 — CPU OVERHEAD AND NIC OFFLOAD SENSITIVITY.

Scenario	Throughput	%idle	%soft	CPU/Gbps
RQ1 Direct	13.879 Gbps	82.1%	0.604%	1.290%
RQ1 VXLAN	9.379 Gbps	85.0%	0.705%	1.597%
Direct (ON)	12.830 Gbps	80.8%	0.901%	1.497%
VXLAN (ON)	8.405 Gbps	85.7%	0.761%	1.702%
Direct (OFF)	3.819 Gbps	83.7%	2.702%	4.273%
VXLAN (OFF)	*failed*	99.4%	0.003%	—

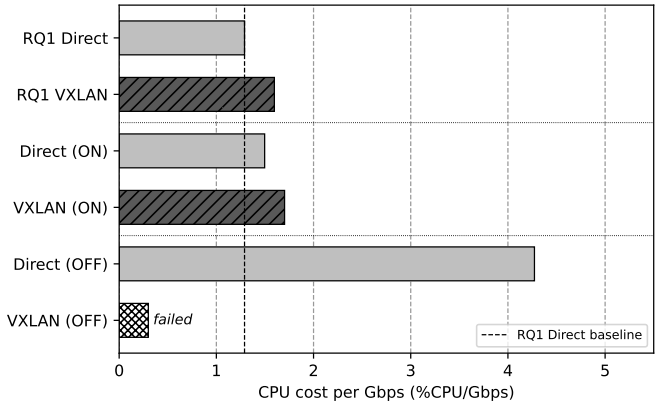

Fig. 5. RQ6 — CPU cost per Gbps across all offload scenarios. The dashed line marks the RQ1 direct baseline (1.290 %CPU/Gbps). VXLAN (OFF) failed to establish a TCP connection; no throughput was recorded.

With offloads enabled, VXLAN requires 1.702 %CPU per Gbps compared to 1.497 %CPU per Gbps for direct traffic, a difference of approximately 13.7%. The `%soft` values (0.761% vs 0.901%) confirm that a measurable fraction of this cost manifests as softirq processing, where the kernel performs network encapsulation.

Disabling sender-side GSO and TSO offloads reduced direct throughput by 70.2% (from 12.830 to 3.819 Gbps) and increased `%soft` from 0.901% to 2.702%, reflecting the additional interrupt processing required when the kernel must handle each packet individually. For VXLAN, disabling offloads caused complete TCP connection failure: all five repetitions timed out, and CPU idle reached 99.4%, confirming that no traffic was processed. This result demonstrates that software VXLAN encapsulation in this environment is critically dependent on sender-side GSO/TSO offloads for TCP connectivity. Without them, the encapsulation overhead prevents the TCP handshake from completing within the connection timeout.

This finding is consistent with the analysis by Pepelnjak, who notes that anything interfering with TCP offload capabilities will significantly degrade forwarding performance in hypervisor-based overlay networking [13]. In environments where offloads must be disabled for security or compatibility reasons, alternative encapsulation approaches or hardware-assisted VTEPs should be considered.

VII. CONCLUSION

This paper provisioned, configured, and experimentally evaluated a VXLAN overlay in a controlled single-host virtualized environment, comparing encapsulated traffic against a direct baseline across six research questions. The results quantify the performance cost of software-based VXLAN encapsulation and reveal several non-trivial interactions between the encapsulation layer and the underlying networking stack.

The baseline TCP evaluation (RQ1) shows that software VXLAN encapsulation reduces throughput by 32.4% relative to direct traffic (13.879 vs 9.379 Gbps) and increases CPU cost by approximately 23.8% per Gbps, confirming that the dominant overhead is computational rather than bandwidth-related. Latency overhead is modest: the overlay adds only 0.060 ms to the average round-trip time (+15%), with lower variability than the direct path. Under UDP (RQ2), the throughput overhead is comparable (28.3%), but packet loss rises sharply from 0.12% to 8.43%, demonstrating that TCP's congestion control absorbs the encapsulation cost gracefully while UDP exposes it as packet loss.

The segment size experiment (RQ3) reveals that, with Generic Segmentation Offload active, the requested MSS has limited effect on on-wire packet sizes, as GSO aggregates segments into 64 KB super-frames regardless of the `-M` parameter. The observable trend, overhead increasing from 19.2% at MSS = 256 B to 28.5% at MSS = 1400 B, reflects the interaction between GSO efficiency and the VXLAN MTU constraint rather than raw segment size effects. Enabling jumbo frames (RQ4) reduces the encapsulation overhead from 32.4% to

18.0%, confirming that larger MTUs amortize the fixed 50-byte encapsulation cost over more payload bytes and reduce the total number of encapsulation operations.

The parallel stream experiment (RQ5) yields the most counter-intuitive finding: with two or more streams, VXLAN consistently outperforms direct traffic, reaching up to 14.925 Gbps against 8.187 Gbps for direct at $P = 4$. This behavior is attributable to Receive Side Scaling: VXLAN encapsulates each stream into a separate outer UDP flow with distinct source ports, providing the per-flow entropy that allows the VMware vSwitch to distribute receive processing across multiple CPU cores.

The CPU and offload evaluation (RQ6) demonstrates that software VXLAN encapsulation in this environment is critically dependent on sender-side GSO and TSO offloads. With offloads enabled, VXLAN requires 1.702 %CPU per Gbps against 1.497 %CPU per Gbps for direct traffic. Disabling offloads reduces direct throughput by 70% and causes complete TCP connection failure for VXLAN, with CPU idle reaching 99.4%, confirming that no traffic was processed. This result has a practical implication: in deployments where offloads must be disabled, software VTEPs may become inoperational, and hardware-assisted encapsulation or alternative overlay mechanisms should be considered.

Limitations

Several limitations of this study warrant acknowledgment. First, the single-host topology eliminates physical-network variability by design, but this also means that results may not generalize directly to multi-host deployments where underlay routing, switch queuing, and physical NIC behavior play a role. Second, the GSO interaction in RQ3 prevented a clean measurement of the raw MSS effect; disabling GSO caused VXLAN TCP connectivity to fail in this environment, suggesting that the encapsulation path is sensitive to offload configuration in ways that merit further investigation. Third, the RQ6 offload test was limited to the sender side only, as disabling receiver-side GRO also caused connectivity failure; a bilateral offload test was therefore not achievable in this setup.

REFERENCES

[1] IEEE, "IEEE Standard for Local and Metropolitan Area Networks—Bridges and Bridged Networks," IEEE Std 802.1Q-2022, 2022.

[2] M. Mahalingam, D. Dutt, K. Duda, P. Agarwal, L. Kreeger, T. Sridhar, M. Bursell, and C. Wright, "Virtual eXtensible Local Area Network (VXLAN): A Framework for Overlaying Virtualized Layer 2 Networks over Layer 3 Networks," RFC 7348, IETF, Aug. 2014.

[3] K. Hamzeh, G. Pall, W. Verthein, J. Taarud, W. Little, and G. Zorn, "Point-to-Point Tunneling Protocol (PPTP)," RFC 2637, IETF, July 1999.

[4] A. Valencia, M. Littlewood, and T. Kolar, "Cisco Layer Two Forwarding (Protocol) 'L2F'," RFC 2341, IETF, May 1998.

[5] W. Townsley, A. Valencia, A. Rubens, G. Pall, G. Zorn, and B. Palter, "Layer Two Tunneling Protocol 'L2TP'," RFC 2661, IETF, Aug. 1999.

[6] J. Lau, M. Townsley, and I. Goyret, "Layer Two Tunneling Protocol - Version 3 (L2TPv3)," RFC 3931, IETF, Mar. 2005.

[7] P. Garg and Y. Wang, "NVGRE: Network Virtualization Using Generic Routing Encapsulation," RFC 7637, IETF, Sept. 2015.

[8] J. Gross, I. Ganga, and T. Sridhar, "Geneve: Generic Network Virtualization Encapsulation," RFC 8926, IETF, Nov. 2020.

[9] A. Sajassi, J. Drake, N. Bitar, R. Shekhar, J. Uttaro, and W. Henderickx, "A Network Virtualization Overlay Solution Using Ethernet VPN (EVPN)," RFC 8365, IETF, Mar. 2018.

[10] T. Muhammad, "Overlay Network Technologies in SDN: Evaluating Performance and Scalability of VXLAN and GENEVE," *International Journal of Computer Science and Technology (IJCST)*, vol. 5, no. 1, 2021.

[11] H. Saeed *et al.*, "Comparative Evaluation of VXLAN with Traditional Overlay Network Protocols," *Indonesian Journal of Computer Science*, vol. 13, no. 2, 2024.

[12] N. Lineogi, "VXLAN Overlay with Two Linux Hosts," *GitHub — azure-cross-solution-network-architectures*, 2021. [Online]. Available: https://github.com/nehalineogi/azure-cross-solution-network-architectures/blob/main/advanced-linux-networking/linux-vxlan.md

[13] I. Pepelnjak, "Performance of Hypervisor-Based Overlay Virtual Networking," *ipSpace.net Blog*, Feb. 2015. [Online]. Available: https://blog.ipspace.net/2015/02/performance-of-hypervisor-based-overlay/

[14] M. Gómez, "Maximizing Performance in VXLAN Overlay Networks," *Telefónica Engineering Blog*, Jan. 2017. [Online]. Available: https://medium.com/@TelefonicaEng/maximizing-performance-in-vxlan-overlay-networks-ec35ebe29440